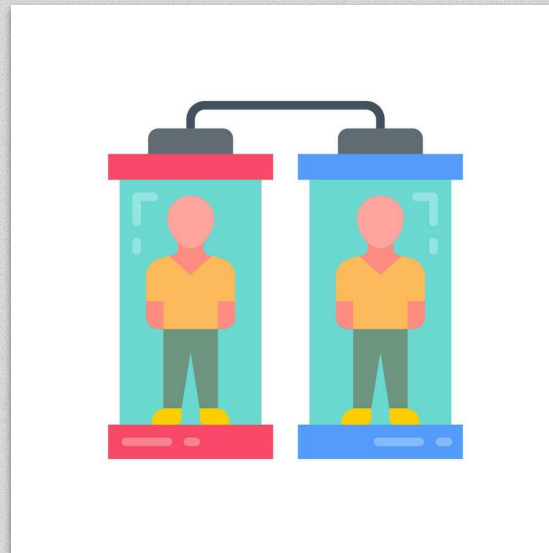


Prototyp



Bartłomiej Handziak



Cel wzorca projektowego

- ◇ Kreacyjny wzorzec projektowy
- ◇ **Pozwala tworzyć kopie istniejących obiektów** bez powiązania kodu z konkretnymi klasami tych obiektów



Opis problemu

- ◇ **Potrzeba uzyskania dokładnej (płytkiej lub głębokiej) kopii danego obiektu**
- ◇ Skopiowanie wszystkich pól oryginału do pól kopii
- ◇ Problem - nie wszystkie pola są dostępne np. pola prywatne, brak wiedzy o wszystkich polach
- ◇ Problem - kod zależny od kopiowanego obiektu, nie jest uniwersalny



Prototyp

- ◇ Prototyp obsługuje kopiowanie instancji klas dziedzicznych
- ◇ Rejestr prototypów



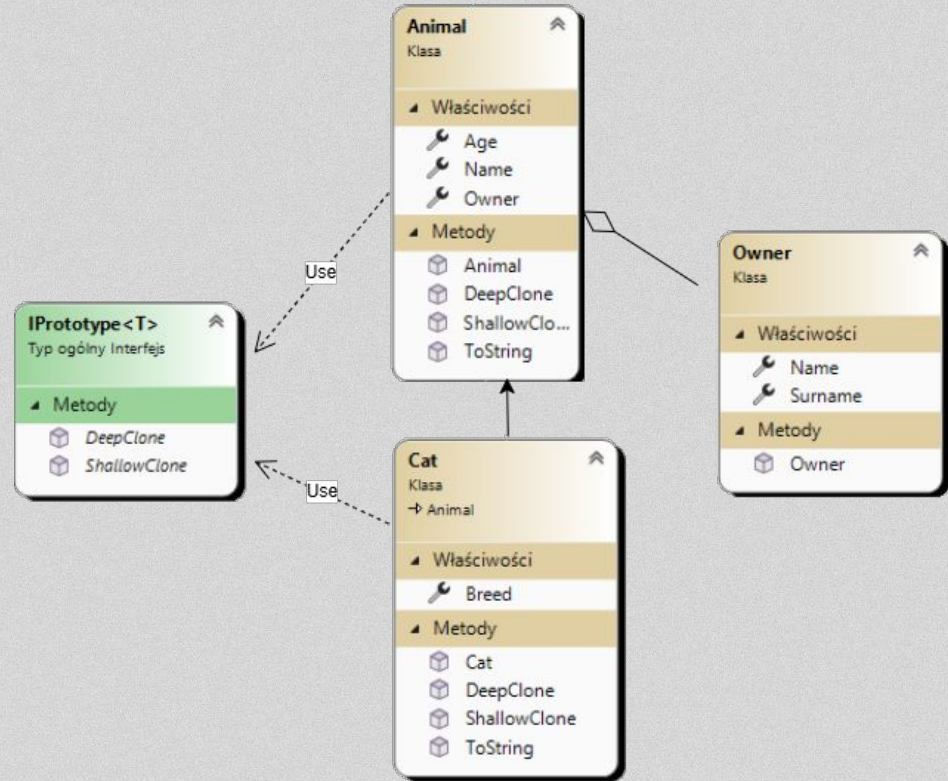
Rozwiązanie - implementacja prototypu

- ◇ Implementacja interfejsu IPrototype
- ◇ Zawiera metody ShallowClone() i DeepClone()
- ◇ Metody tworzą obiekt swojej klasy i kopiują do niej wartości jej pól.
- ◇ Można wtedy skopiować pola prywatne, ponieważ jest do nich dostęp wewnątrz klasy np. za pomocą wskaźnika *this*



Schemat UML

Obiekt posiadający
funkcjonalność klonowania
nazywa się prototypem



Odwołania: 4
`public interface IPrototype<T>`
`{`
 `Odwołania: 4`
 `T ShallowClone();`
 `Odwołania: 3`
 `T DeepClone();`
`}`

Definicja interfejsu generycznego

Odwołania: 4
`public virtual Animal ShallowClone()`
`{`
 `return (Animal) this.MemberwiseClone();`
`}`

Odwołania: 3
`public virtual Animal DeepClone()`
`{`
 `Animal clone = (Animal) this.MemberwiseClone();`
 `clone.Owner = new Owner(Owner.Name, Owner.Surname);`
 `return clone;`
`}`

Implementacja metod klonujących dla klasy Animal

```

static void Main(string[] args)
{
    Owner janKowalski = new Owner("Jan", "Kowalski");
    Animal animal = new Animal("Azor", 4, janKowalski);
    Cat cat = new Cat("Puszek", 6, janKowalski, "Norweski leśny");

    // kopia płytka
    Animal animalCopy = animal.ShallowClone();

    animalCopy.Age = 1;
    animalCopy.Name = "Reks";
    animalCopy.Owner.Name = "Staszek";

    // kopia głęboka
    Animal animalDeepCopy = animal.DeepClone();

    animalDeepCopy.Age = 1;
    animalDeepCopy.Name = "Reks";
    animalDeepCopy.Owner.Name = "Andrzej";

    // przykład dla klasy dziedzicznej
    Cat catCopy = cat.ShallowClone();
    catCopy.Age = 2;
    catCopy.Name = "Kicia";
    catCopy.Breed = "Mainkun";

    Console.WriteLine("Oryginał animal: ");
    Console.WriteLine(animal);

    Console.WriteLine("Kopia płytka animal: ");
    Console.WriteLine(animalCopy);

    Console.WriteLine("Kopia głęboka animal: ");
    Console.WriteLine(animalDeepCopy);

    Console.WriteLine();

    Console.WriteLine("Oryginał cat: ");
    Console.WriteLine(cat);
    Console.WriteLine("Kopia płytka cat: ");
    Console.WriteLine(catCopy);
}

```

Oryginał animal:

Name: Azor, age: 4, owner: Staszek Kowalski

Kopia płytka animal:

Name: Reks, age: 1, owner: Staszek Kowalski

Kopia głęboka animal:

Name: Reks, age: 1, owner: Andrzej Kowalski

Oryginał cat:

Name: Puszek, age: 6, owner: Staszek Kowalski, breed: Norweski leśny

Kopia płytka cat:

Name: Kicia, age: 2, owner: Staszek Kowalski, breed: Mainkun

Podsumowanie

- ◇ Kreacyjny wzorzec projektowy
- ◇ Pozwala na klonowanie istniejących obiektów
- ◇ Używa metod: `ShallowClone()`, `DeepClone()`
- ◇ Pozwala uniknąć zależności od konkretnych klas

